Leveraging LLM to Generate Unittests for Notifiers in Taskflow

SNIKITHA SIDDAVATAM





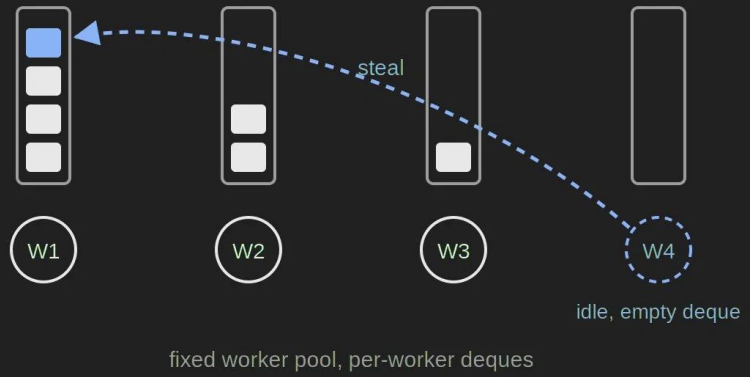
Takeaways

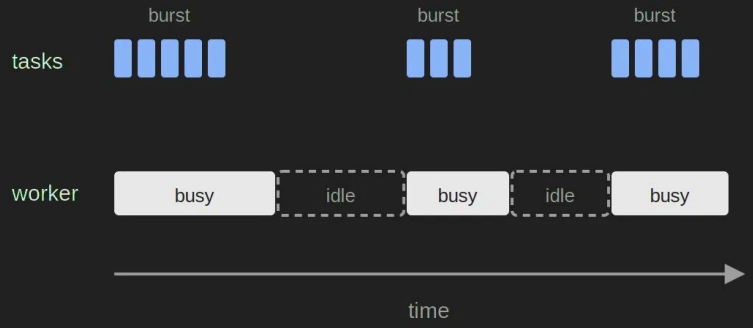
- Understand the importance of Notifiers
- Understand how Notifier is used in Taskflow
- Recognize the challenges of testing a Notifier
- How existing testing framework works
- Present LLM-assisted solution
- Demonstrate experimental results

We found a bug that shipped in a production scheduler for over an year and an LLM helped write the test that caught it.

Task-Parallel Systems

- Intel TBB[1], Taskflow[2], folly: work-stealing schedulers with a fixed worker pool
- Tasks come and go in bursts; workers alternate between busy and idle
- Idle workers must not spin (wasted cores, power) and must not oversleep (latency, deadlock)





[1]:https://www.intel.com/content/www/us/en/developer/tools/oneapi/onetbb.html

[2]:https://github.com/taskflow/taskflow

Task-Parallel Systems



spin

wasted cores, power



wake exactly when work arrives

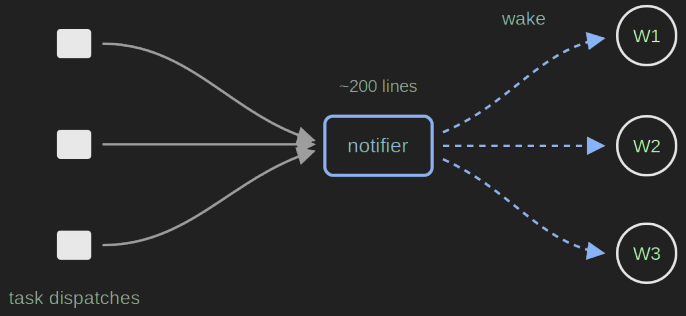


oversleep

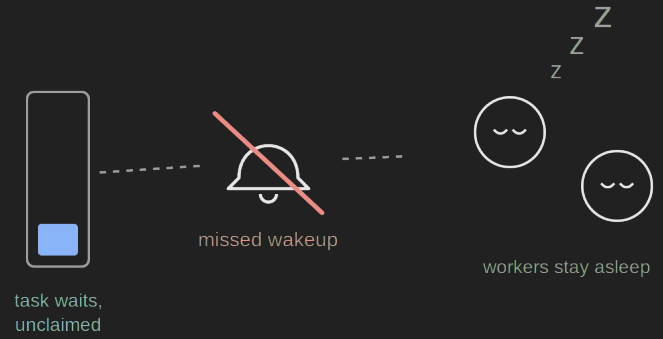
latency, deadlock

Notifiers

- The primitive that puts workers to sleep and wakes them when work arrives
- Small code, huge blast radius: every task dispatch flows through it
- One missed wakeup → a task sits unclaimed while workers sleep → throughput collapse or hang



small code, every dispatch flows through it



→ throughput collapse or hang

Taskflow



- C++ header-only task graph library, DAG-based parallelism
- tf::Executor = work-stealing thread pool

Taskflow

Ubuntu passing | macOS passing | Windows passing | API docs | profiler tfp | cite TPDS

Taskflow helps you quickly write high-performance task-parallel programs with high programming productivity. It is faster, more expressive, fewer lines of code, and easier for drop-in integration than many existing task programming libraries.

Start Your First Taskflow Program

The following program (`simple.cpp`) creates four tasks `A` , `B` , `C` , and `D` , where `A` runs before `B` and `C` , and `D` runs after `B` and `C` . When `A` finishes, `B` and `C` run in parallel. Try it live on Compiler Explorer!

```cpp
#include <taskflow/taskflow.hpp>  // Taskflow is header-only

int main(){

  tf::Executor executor;
  tf::Taskflow taskflow;

  auto [A, B, C, D] = taskflow.emplace(  // create four tasks
    [] () { std::cout << "TaskA\n"; },
    [] () { std::cout << "TaskB\n"; },
    [] () { std::cout << "TaskC\n"; },
    [] () { std::cout << "TaskD\n"; }
  );

  A.precede(B, C);  // A runs before B and C
  D.succeed(B, C);  // D runs after  B and C

  executor.run(taskflow).wait();

  return 0;
}
```

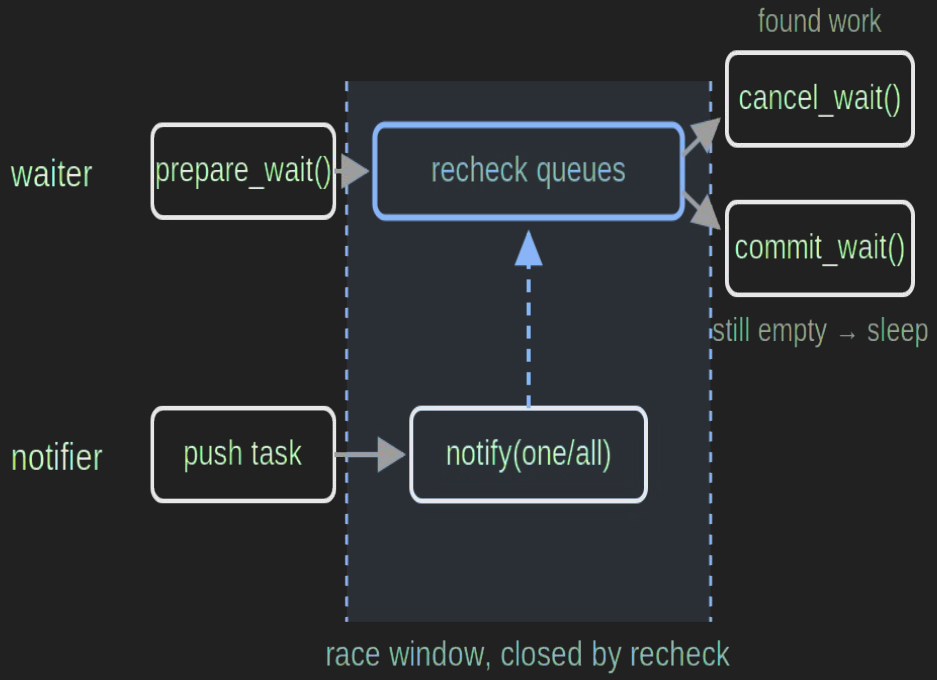
https://github.com/taskflow/taskflow

Where the Notifier Sits

- One notifier per executor. Every worker waits on it.
- Every task dispatch calls notify_one()
- Every worker that runs out of work calls prepare_wait()
- Before it sleeps, it rechecks all the queues one more time
- Taskflow ships two of them, so we test both

The Two-Phase Wait Protocol

- Waiter: prepare_wait() → recheck queues → commit_wait() or cancel_wait()
- Notifier side: push task → notify(one/all)
- The recheck between prepare and commit is what closes the race window (in theory)



The Two Notifiers

Non-blocking Notifier

```
struct Waiter {

  alignas(TF_CACHELINE_SIZE) std::atomic<Waiter*> next;   // intrusive stack

  uint64_t epoch;                                         // admission ticket

  enum : unsigned { kNotSignaled = 0, kWaiting, kSignaled };

  std::atomic<unsigned> state {0};                        // per-waiter automaton

};
```

Three mechanisms are hand-implemented:

- an intrusive lock-free stack of parked waiters
- a ticket discipline ordering waiters by admission
- a three-state automaton per waiter, with explicit _park / _unpark

Atomic Notifier

```
void prepare_wait(size_t w) noexcept {

  auto prev = _state.fetch_add(WAITER_INC,
std::memory_order_relaxed);

  _waiters[w].epoch = (prev >> EPOCH_SHIFT);


std::atomic_thread_fence(std::memory_order_seq_cst);
// (F1)

}

void cancel_wait(size_t) noexcept {

  _state.fetch_sub(WAITER_INC,
std::memory_order_relaxed);

}
```

```
void commit_wait(size_t w) noexcept {

  uint64_t prev =
_state.load(std::memory_order_relaxed);

  while ((prev >> EPOCH_SHIFT) ==
_waiters[w].epoch) {

    _state.wait(prev,
std::memory_order_relaxed);

    prev =
_state.load(std::memory_order_relaxed);

  }

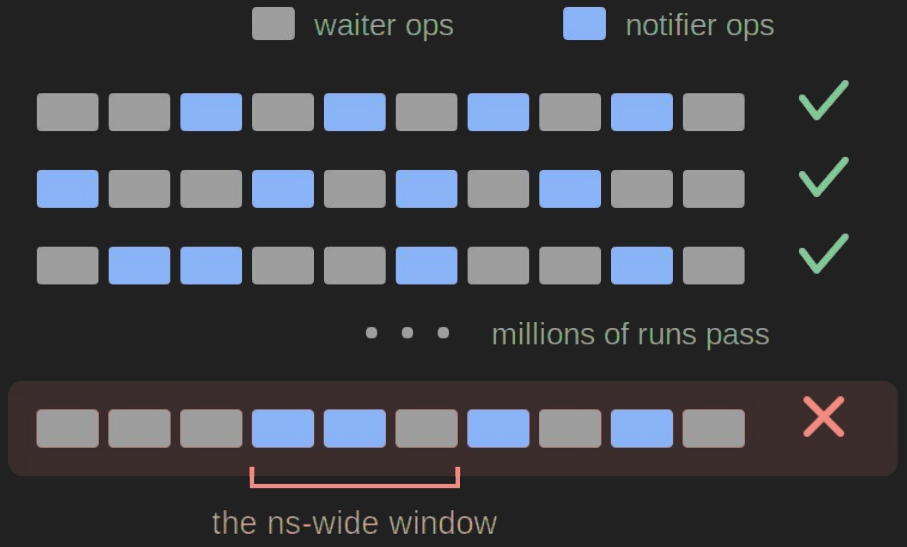
  _state.fetch_sub(WAITER_INC,
std::memory_order_relaxed);

}
```

Why is it so difficult to test

1. The window is nanoseconds wide - bug only fires on one specific interleaving; scheduler rarely produces it; survives millions of stress-test runs;

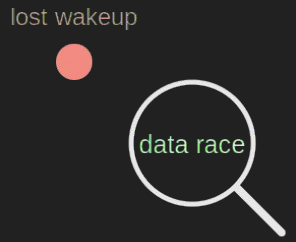
   can't reproduce = can't debug



one ordering fires the bug — can't reproduce, can't debug

Why is it so difficult to test

2. Existing tools miss it - TSan flags data races, not lost wakeups (it's a semantic bug, every access is "properly" atomic); stress testing is probabilistic and flaky; formal verification doesn't scale to real C++ with memory orderings



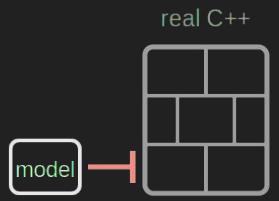
TSan

semantic bug,
every access "atomic"



stress testing

probabilistic,
flaky

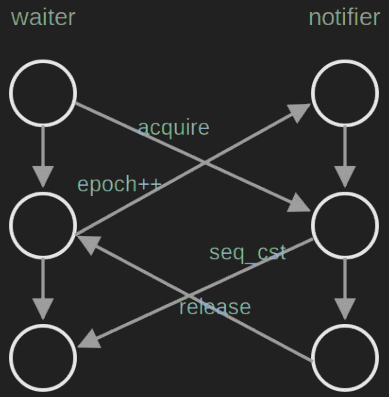


formal verification

doesn't scale to
memory orderings

Why is it so difficult to test

3. Human reasoning doesn't scale - correctness depends on acquire/release orderings, epoch counters, and a cross-thread state machine; the proof lives in a comment; only a handful of experts can review it



cross-thread state machine

```
// proof sketch:
// if notify sees no
// waiter, waiter must
// see the task...
```

the proof lives in a comment



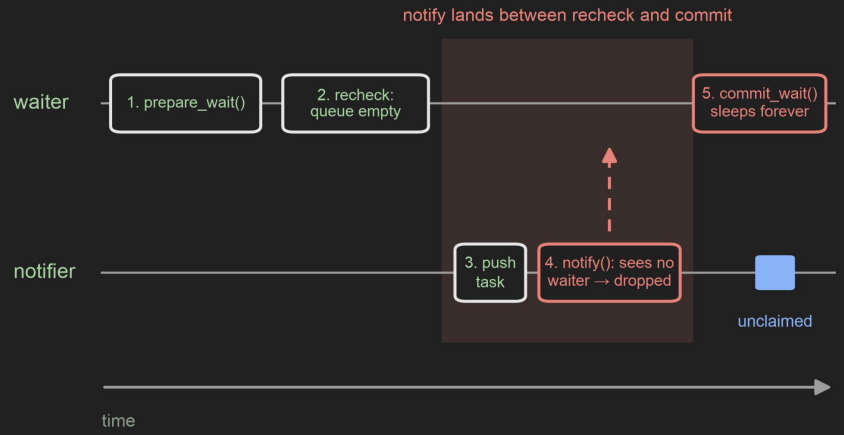
a handful of reviewers

And They Keep Coming Back

- Notifiers evolve: memory ordering tweaks, refactors, ports across systems
- Every change demands a new round of expert-crafted tests
- Expertise-heavy, tedious, easy to skip under deadline pressure

What Goes Wrong

- Lost wakeup: notify lands between recheck and commit, state update misordered → waiter sleeps forever
- Spurious wake / double notify: correctness okay, performance not
- Interleaving timeline diagram: two threads, the exact bad ordering

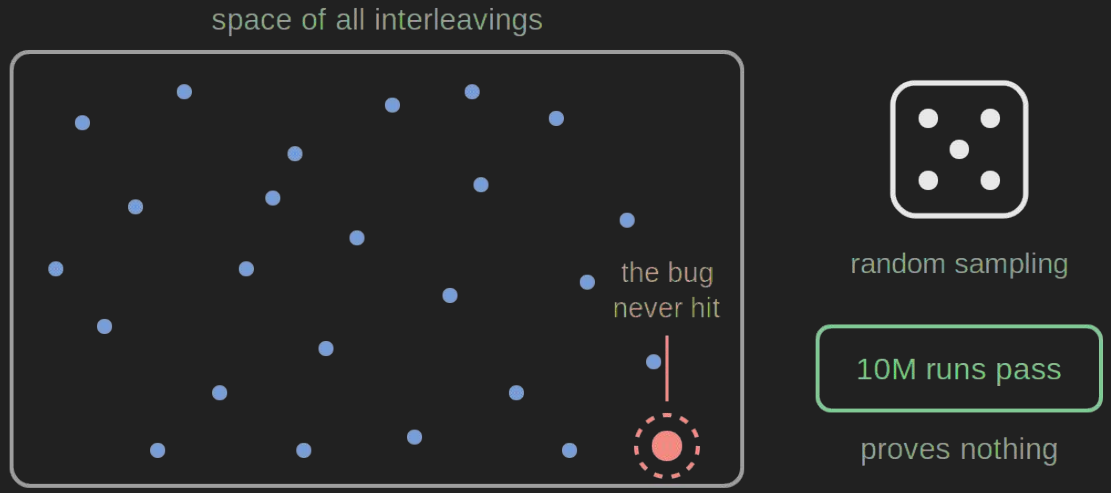




correctness okay — wasted wakeups, performance not

Existing Tool #1: Stress Testing

- Spawn threads, hammer wait/notify in a loop, hope
- Cheap, but probabilistic - a pass proves nothing
- No control over which interleavings you actually explored



Existing Tool #2: ThreadSanitizer

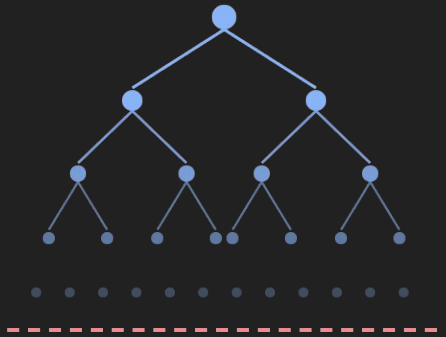
- Detects data races on executions that actually happen
- Lost wakeups can be race-free → TSan sees nothing
- Can't reason about liveness ("this thread should eventually wake")



Existing Tool #3: Model Checking

- Exhaustive interleaving exploration (SPIN, GenMC, loom-style)
- Sound - but requires rewriting code into the checker's model
- State explosion caps it at tiny fragments; nobody model-checks production Taskflow

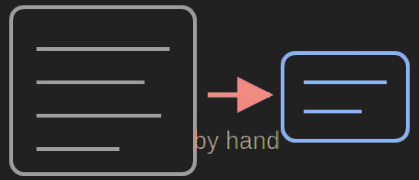
exhaustive exploration



state explosion

sound, but only for tiny fragments

requires a rewrite



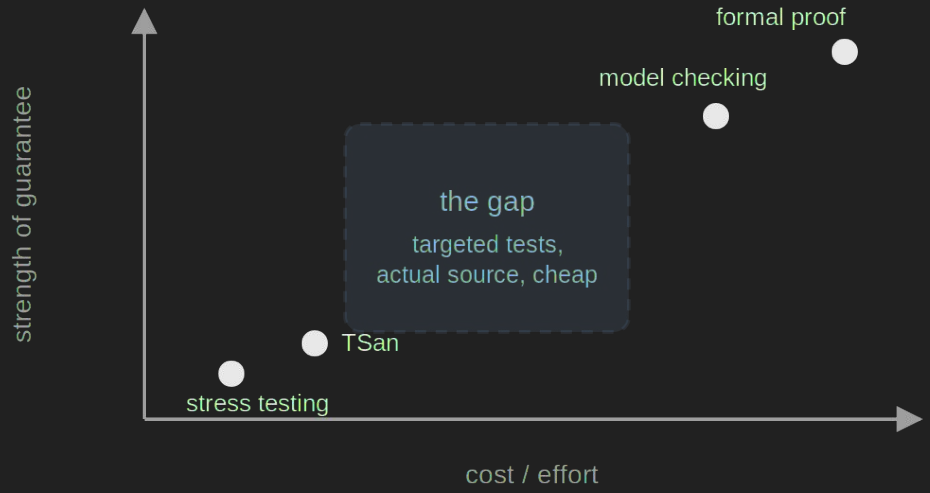
real Taskflow C++ checker model

what you verify isn't
what you ship

nobody model-checks
production Taskflow

The Gap

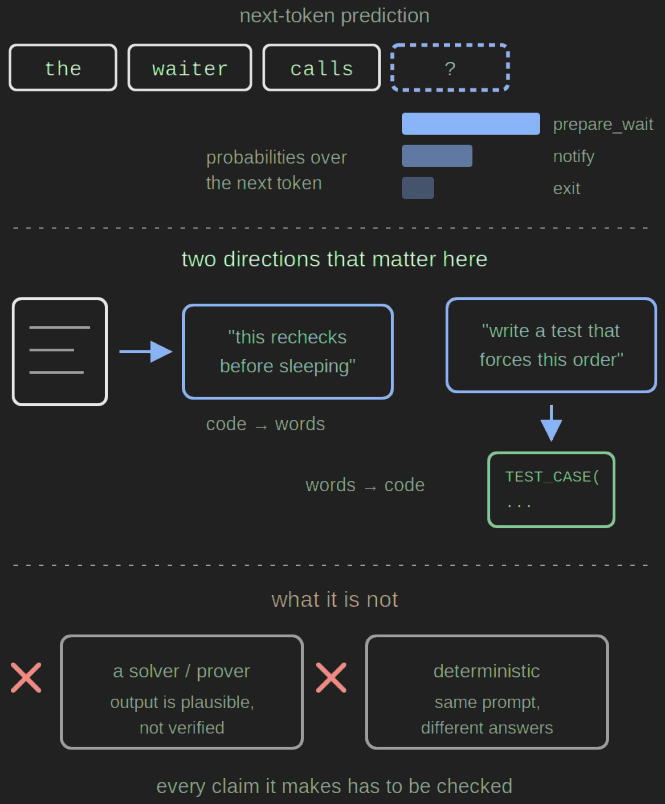
- Table: cost vs. guarantee for stress / TSan / model checking / formal proof
- Nothing practical in the middle for real production C++
- What we want: targeted adversarial tests, written against the actual source, cheaply



approach	cost	guarantee	runs on real code?
stress testing	low	none (probabilistic)	yes
TSan	low	data races only	yes
model checking	high	sound	rewritten model
formal proof	very high	strongest	no
what we want	low	targeted, adversarial	yes

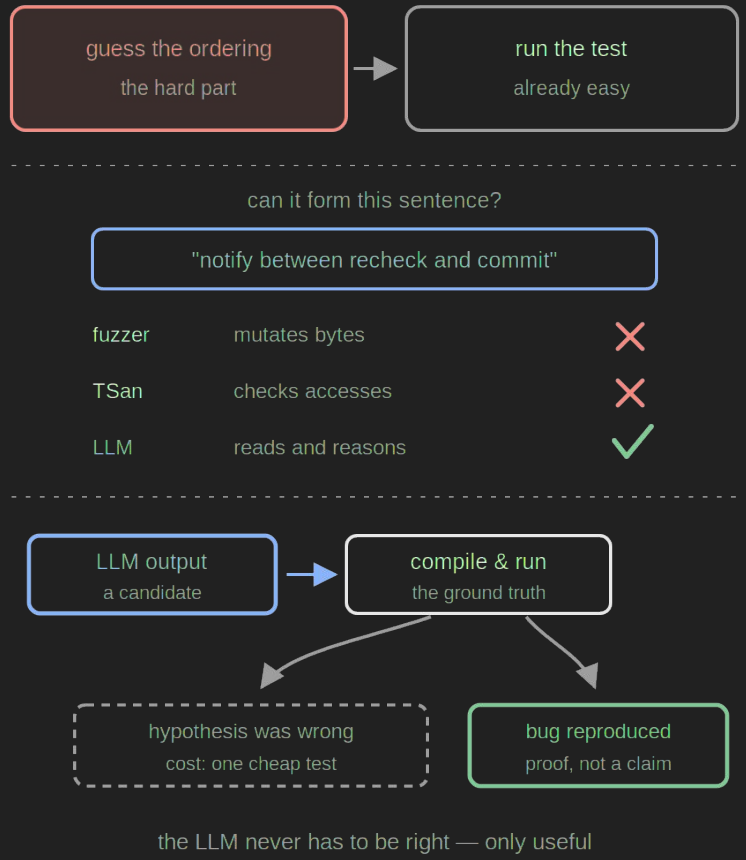
What is a Large Language Model (LLM)?

- A model trained on huge amounts of text and code; predicts the next token given everything before it
- Practically: reads a file the way you would, and writes text back - prose, C++, whatever
- Two things it's good at that matter here: explaining what code does in words, and writing code from a description in words
- Not a solver, not a prover: it produces plausible output, so every claim needs checking
- Non-deterministic - same prompt can give different answers



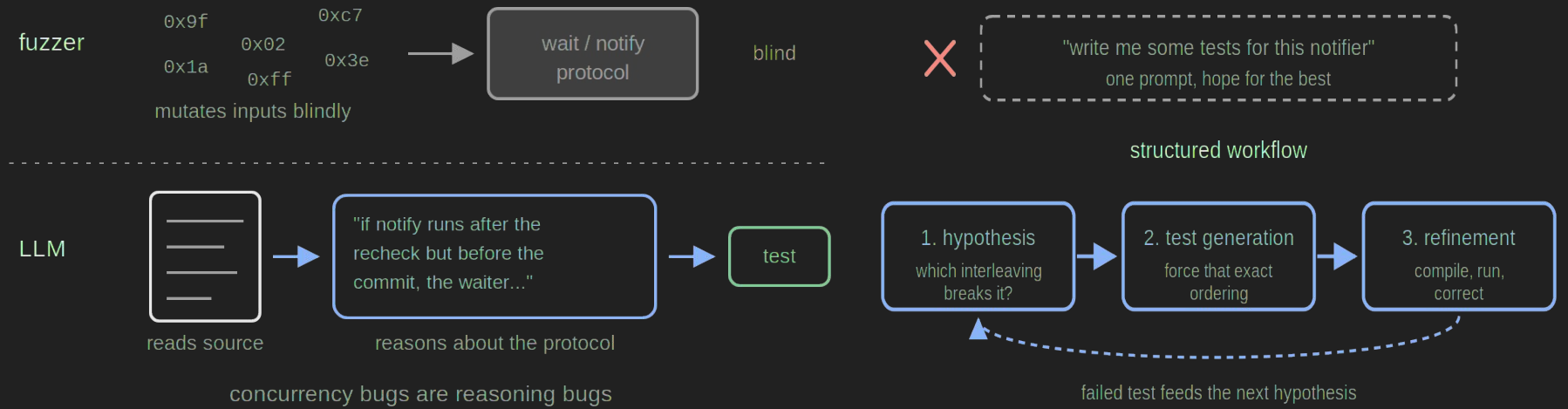
Why that helps with a notifier?

- The hard part of a lost wakeup isn't running the test, it's guessing which ordering to test
- That guess is a reading-comprehension task over ~200 lines of atomics - words in, words out
- Fuzzers mutate bytes; TSan checks accesses; neither can form the sentence "notify between recheck and commit"
- The LLM's output is a candidate, not an answer - the test either reproduces the bug or it doesn't. Compiler and runtime are the ground truth
- So the unreliability is fine: false hypotheses cost one cheap test each



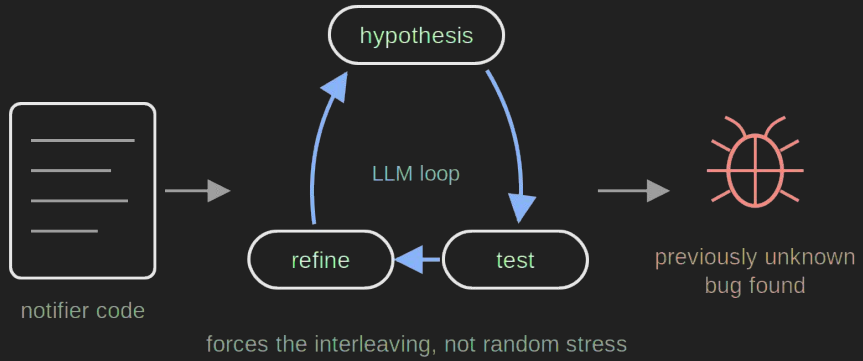
Idea: LLMs as Adversarial Test Writers

- LLMs can read source and reason about protocols in natural language
- Concurrency bugs are reasoning bugs - exactly where fuzzers are blind
- Not "LLM writes tests for free" - LLM guided by a structured workflow



Why I think LLM can help

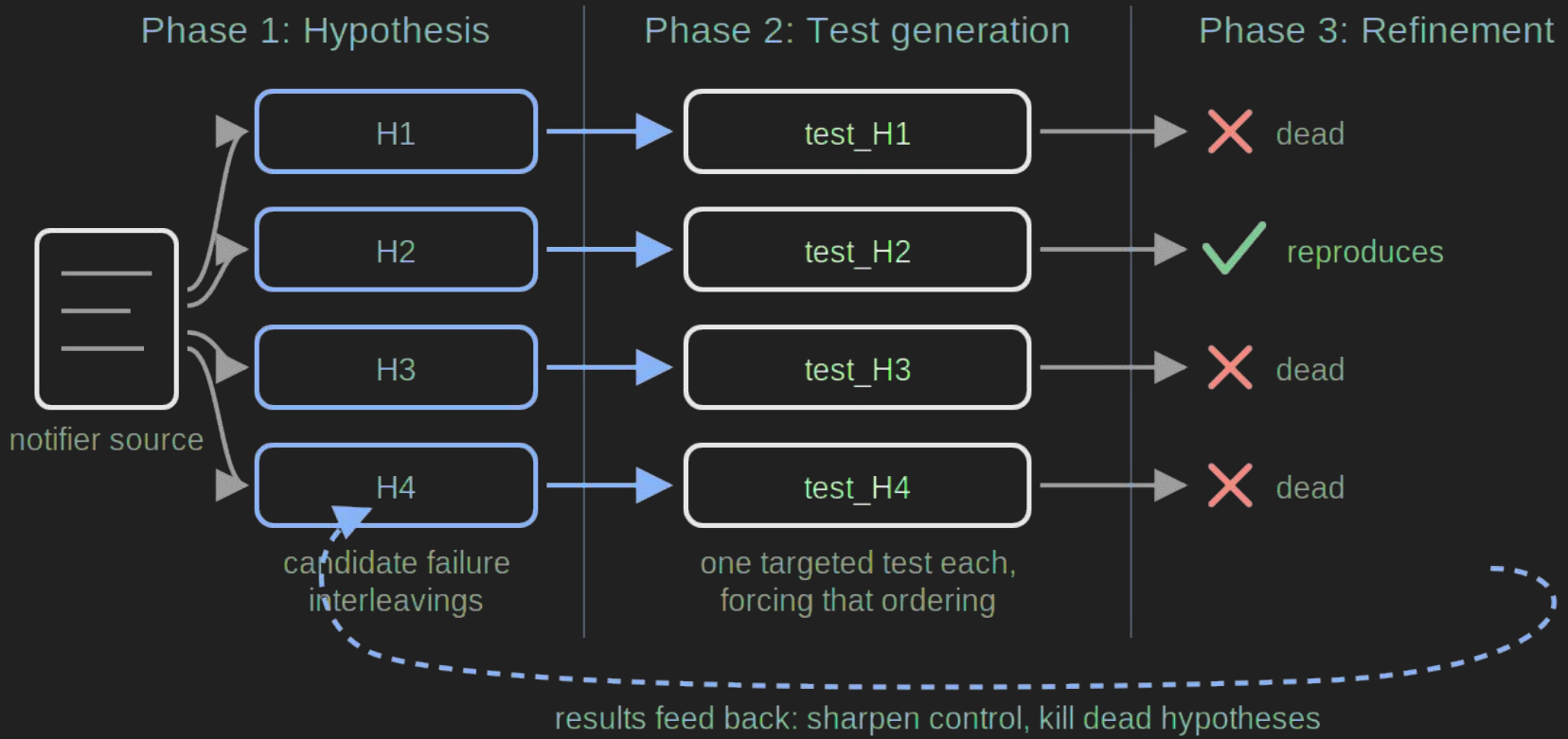
- Reads the notifier code and hypothesizes which interleaving breaks it
- Turns the hypothesis into a targeted unit test that forces that interleaving - not random stress
- Cheap to iterate: hypothesis → test → refine
- Result: found a previously unknown bug in Taskflow's notifier



The Three-Phase Workflow

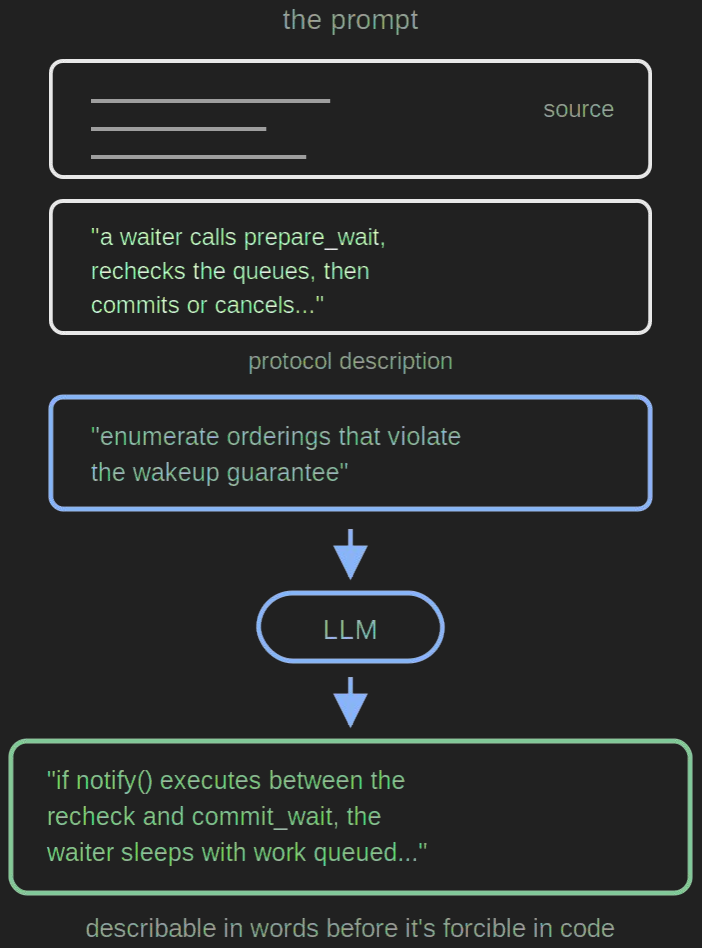
- Phase 1: Hypothesis - LLM reads notifier source, proposes candidate failure interleavings
- Phase 2: Test generation - one targeted unit test per hypothesis, forcing that interleaving
- Phase 3: Refinement - feed results back, sharpen interleaving control, kill dead hypotheses

The Three-Phase Workflow



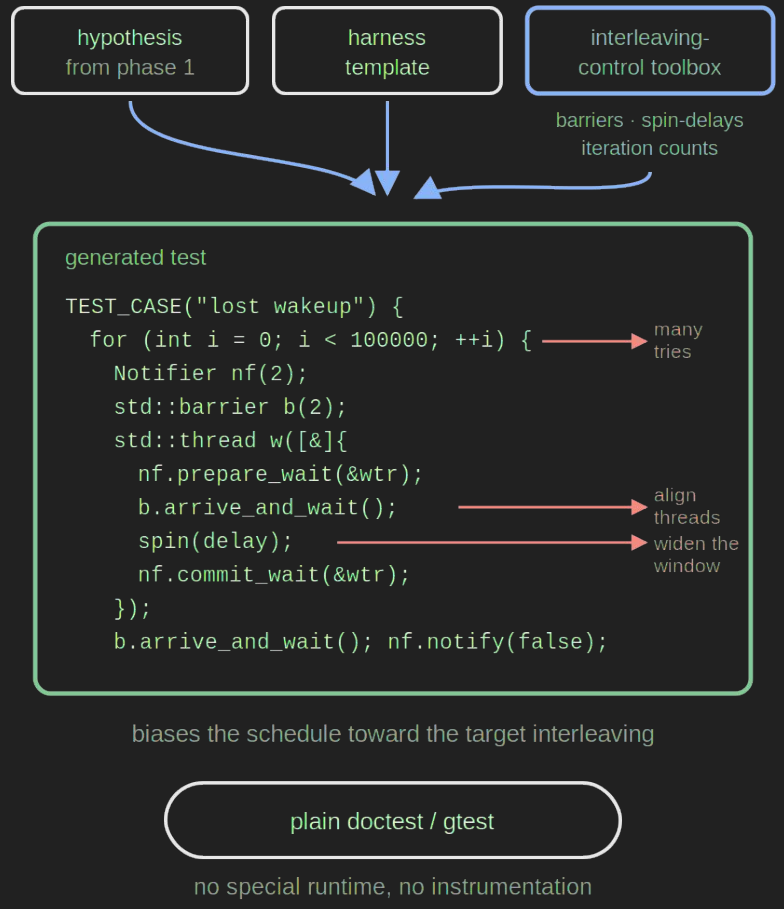
Phase 1: Hypothesis

- Prompt structure: source + protocol description + "enumerate orderings that violate the wakeup guarantee"
- Example real hypothesis output ("if notify() executes between the recheck and commit_wait...")
- Why this works: the bad interleaving is describable in words before it's forcible in code



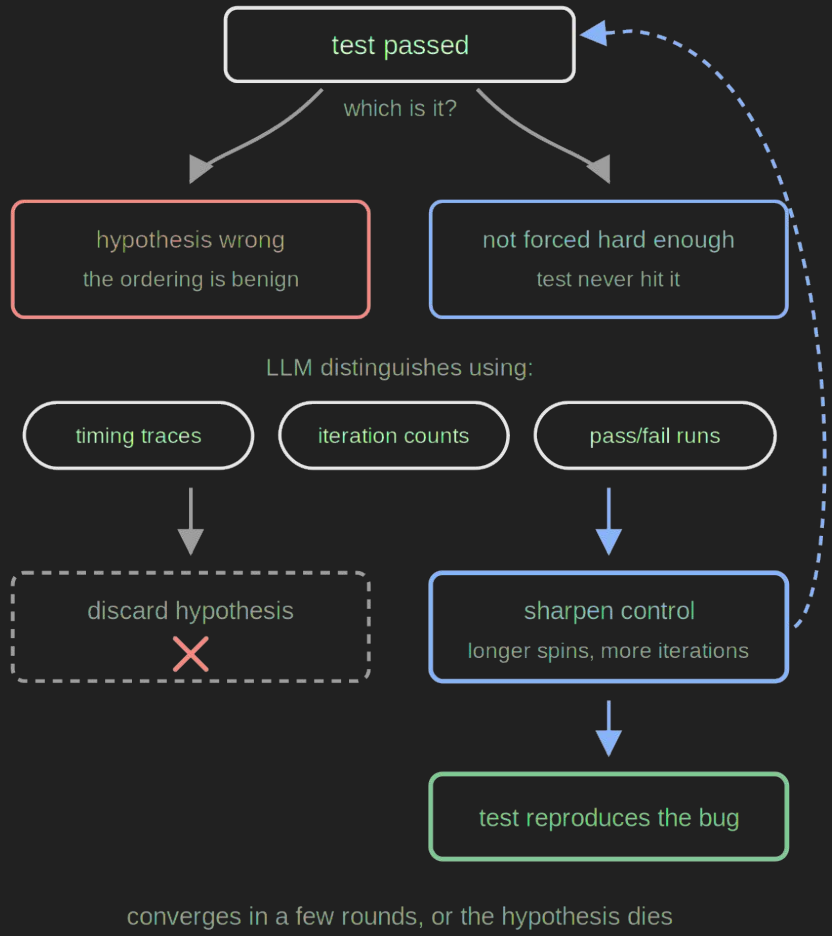
Phase 2: Test Generation

- Prompt: hypothesis + test harness template + interleaving-control toolbox (barriers, spin-delays, iteration counts)
- Show a generated test, annotated: where it biases the schedule toward the target interleaving
- Tests are plain doctest/gtest - no special runtime



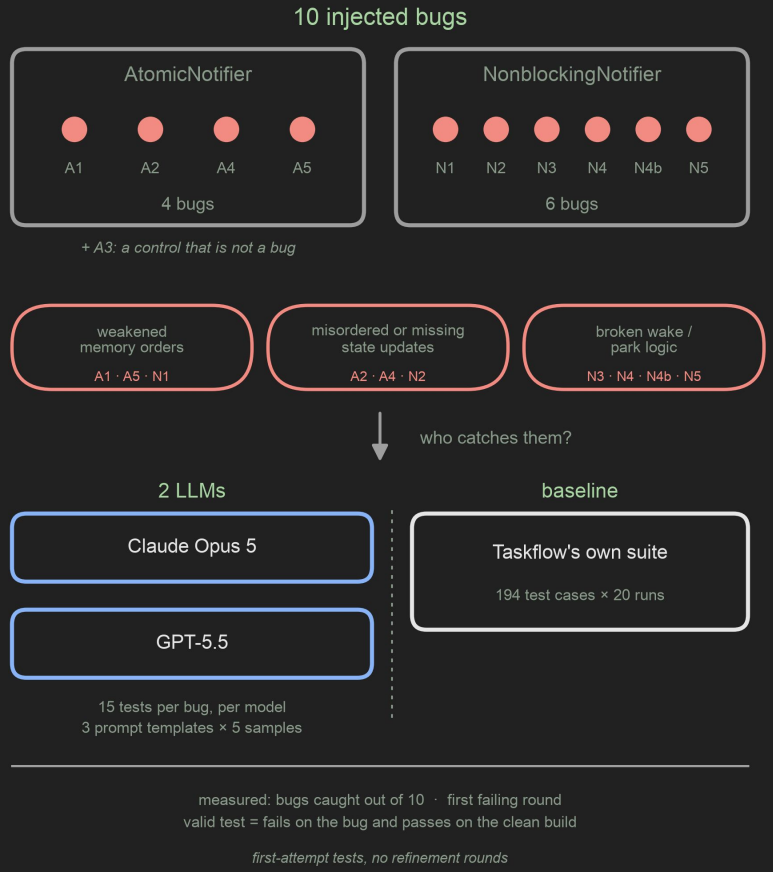
Phase 3: Refinement

- Test passed: hypothesis wrong, or interleaving not forced hard enough? LLM distinguishes
- Feed back: timing traces, iteration counts, pass/fail across runs
- Converges in a few rounds or discards the hypothesis



Evaluation: Controlled Fault Injection

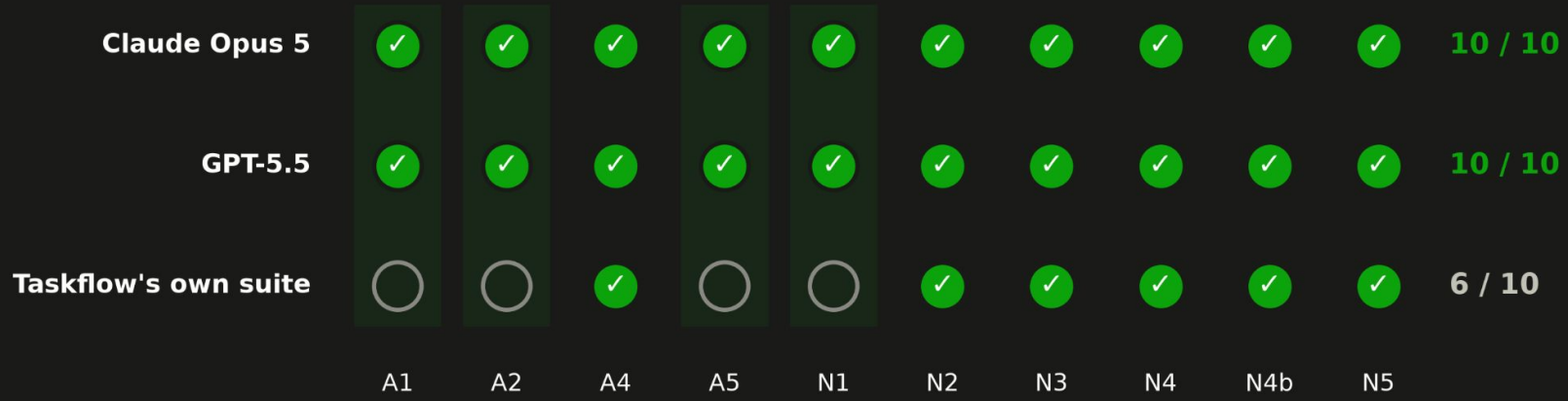
- 10 injected bugs: 5 in AtomicNotifier, 5 in NonblockingNotifier (weakened memory orders, dropped rechecks, off-by-one signal accounting)
- Baselines: human expert-written tests
- 2 LLMs compared



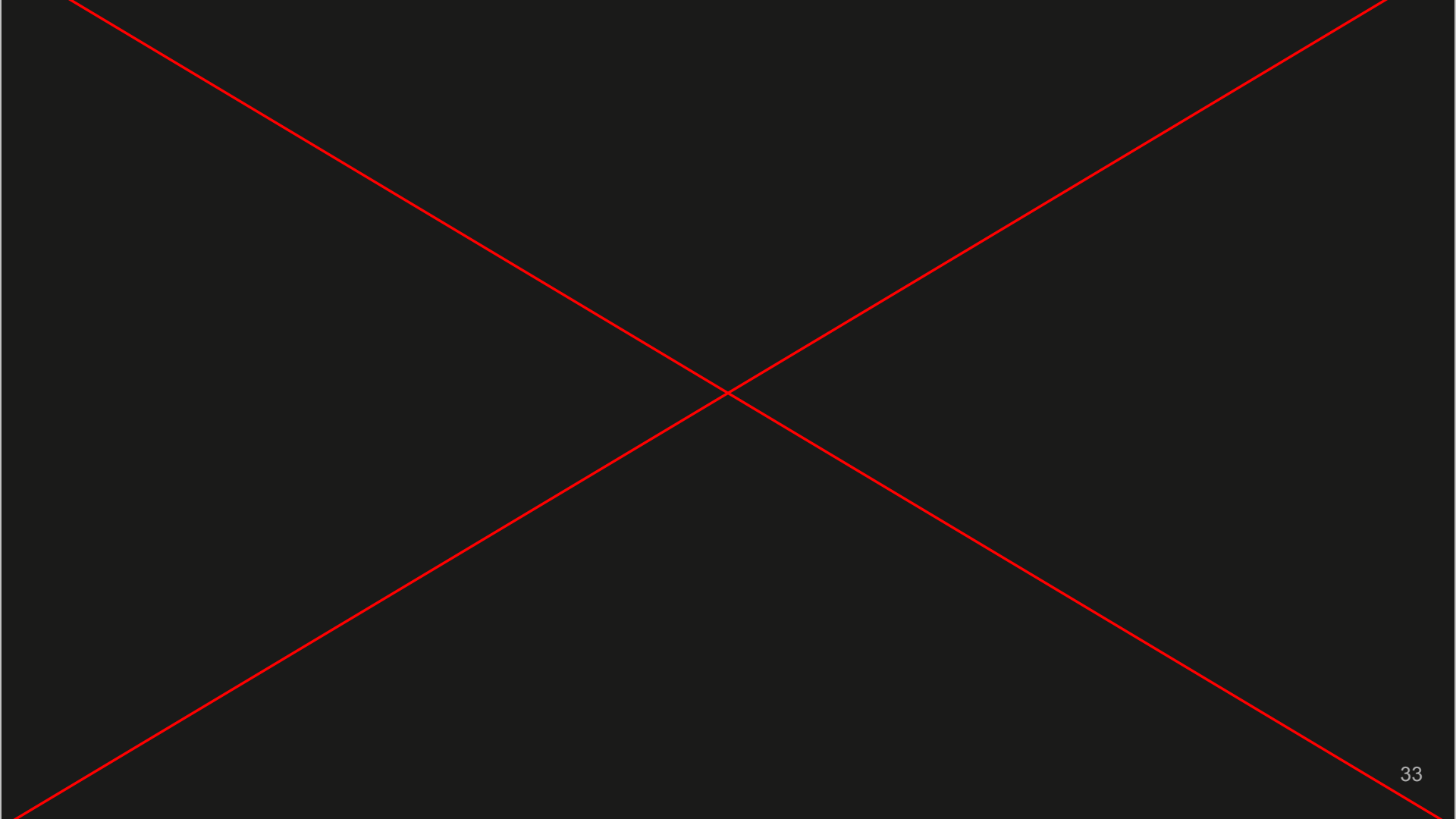
Results

Both models caught four bugs the project's suite misses

Injected concurrency bugs in Taskflow's AtomicNotifier and NonblockingNotifier



The four shaded bugs are memory ordering faults. One weakens a seq_cst fence. Another lets a notify run before the epoch update. Taskflow's suite misses all four, across 194 test cases and 20 runs each, on both arm64 and x86_64.



exp — AtomicNotifier A1 (lost wakeup)

What does an LLM-written test look like?

Four real tests from our experiment. I trimmed them to fit on a slide,

but the logic is the same.

Four tests, four lessons

#	Model	Bug	What happened
1	Claude Opus 5	**A1**	Caught the bug. Quiet on correct code.
2	GPT-5.5	**A5**	Looked right. Missed the bug.
3	Both	**A5**	Both caught it. One was much faster.
4	GPT-5.5	**A5**	Caught the bug, but also failed on correct code.

Every test starts the same way

```
// We gave this to the model. It must copy it unchanged.

[[noreturn]] inline void fail(const char* why) {

  std::fprintf(stderr, "DETECTED: %s\n", why);

  std::exit(1);

}

// Found a bug?  call fail("why")

// No bug?       return 0
```

The rules we gave:

- Compile with no edits.
- Print "ROUND n" each round.
- Stop within 25 seconds.
- Call fail() with a reason.
- Pass on the correct notifier.

Source: exp/prompts/shared_test_rules.md

Three ways of asking

- Prompt 1: "Put all threads to sleep first. Then notify. Use a timeout. Count who wakes up."
- Prompt 2: "Choose threads, rounds and timing to hit the race window. Explain your choices."
- Prompt 3: "Aim at the gap between prepare_wait and commit_wait. Put a notify inside it."

We ran each prompt 5 times for each bug and each model. With the same bug and model, a different prompt often gave a very different test.

Example 1 (Claude, A1, prompt 1): It writes the plan first

```
// Stress test for lost wakeups in tf::AtomicNotifier.
//
// (A) "commit-all": all N threads are driven into the
//     committed waiter state ... Only then ... notify_all()
// (B) "commit-partial": same, but notify_n(k) first ...
// (C) "race": prepare_wait / re-check / commit_wait ...
//     in lockstep, while the main thread ... issues a
//     single notify_all() at a randomized offset
```

Example 1: the waiter threads

```
// 1  wait at the starting line
while(!g_start.load()) cpu_relax();

// 2  wait a small random time
unsigned d = xs(seed) & 127u;
while(d--) cpu_relax();

// 3  the real waiting protocol
if(g_pred.load() < g) {
  NOTIFIER->prepare_wait(id);
  if(g_pred.load() >= g)
    NOTIFIER->cancel_wait(id);
  else                          // 4
    NOTIFIER->commit_wait(id);
}
```

Example 1: the main thread

```
g_start.store(true);                // 1

unsigned d = xs(seed) & 255u;
while(d--) cpu_relax();             // 2

g_pred.store(g);                    // 3
NOTIFIER->notify_all();

if(!wait_for(all_done, 3000ms))     // 4
  fail("lost wakeup: %zu of %zu"    // 5
       " threads still stuck in commit_wait 3s after ...");
```

Example 1: It fails only when the bug is there

```
DETECTED: lost wakeup: 1 of 8 threads still stuck
in commit_wait
3s after the predicate was set and notify_all() was
called
```

Run the same test on…	Result (20 runs each)
Correct notifier	**Passed 20 of 20**
A3, a change that is not a bug	**Passed 20 of 20**
A1, the real bug	**Failed 20 of 20, in round 1**

Example 2 (GPT-5.5, A5, prompt 3): Short, clean, easy to read

```
tf::AtomicNotifier notifier(1);

std::thread waiter([&] {
  notifier.prepare_wait(0);      // 1
  phase.store(1);
  while(phase.load() == 1)       // 2
    std::this_thread::yield();
  notifier.commit_wait(0);       // 4
  done.store(true);
});

while(phase.load() != 1) yield();
notifier.notify_all();           // 3
phase.store(2);
// not done in 5 s? fail()       // 5
```

Source: exp/tests/gpt_A5_2b_3_s3.cpp

Example 2: It passed. It missed the bug.

- This test takes turns. One thread finishes before the other starts, so each thread always sees the other's latest write.
- A5 needs both threads to act at the same moment. Each thread misses the other's write. The waiter goes to sleep, and nobody wakes it.
- The result on A5 was passed 5 of 5 runs. The test also never checks for work, and the bug hides in that check.
- Lesson: easy to read is not the same as good at finding bugs.

Example 3 (Claude vs GPT-5.5, A5): Same idea, very different speed

Claude, one round:

```
for (it = 1; it <= 30000; ++it) {
  p->ready.store(it);                    // "go"
  busy_delay(((it >> 6) & 15u) * 4u);
  if (p->pred.load() != it) {
    p->nf.prepare_wait(0);
    if (p->pred.load() != it)
p->nf.commit_wait(0);
    else
p->nf.cancel_wait(0);
  }
}
```

GPT-5.5, one round:

```
// just once per pair:
p->marker.store(r);                          // "go"
tiny_delay((r*17 + index*13) & 63);
p->notifier.prepare_wait(0);
if (p->predicate.load())
p->notifier.cancel_wait(0);
else
p->notifier.commit_wait(0);
// then main thread checks all pairs, starts
round r+1
```

Sources: claude_A5_2b_1_s1.cpp (prompt 1), gpt_A5_2b_2_s2.cpp (prompt 2)

Example 3: Count tries, not rounds

	Claude Opus 5	GPT-5.5
Tries in one round	30,000 per pair	1 per pair
Waiter/notifier pairs	4	12
First failure	Round 1	Round 41,531
Tries before failure	at most 120,000	about 500,000
Caught A5 in	**5 of 5 runs**	**1 of 5 runs**
Correct notifier	Passed	Passed

Example 4 (GPT-5.5, A5, prompt 2): A test that cannot stop

```
MAX_ROUNDS = 128;                       // 1
ATTEMPTS_PER_ROUND = 1024;

// main thread, every round:
while (finished < expected) {
  shared.finished.wait(cur);            // 2
  if (now() >= hard_deadline)           // 3
    break;
}
// after the loop:
shared.round.store(MAX_ROUNDS);         // 4
for (auto& t : threads) t.join();
```

Source: exp/tests/gpt_A5_2b_2_s5.cpp

A test that fails on everything tells you nothing

- On bug A5: failed 5 of 5 runs, in round 1. It found a real lost wakeup.
- On the correct notifier: hung 5 of 5 runs. That is a false alarm every time.

This is why we also run every test on the correct notifier. For Claude, 35 of 164 tests (21%) failed there, so we threw them out.

What the good tests had in common

Pattern	Why it helps
A starting gate	Threads really run at the same moment.
Small changing delays	Each try hits a slightly different moment.
Many tries per round	The rare timing shows up before time runs out.
A timer	A lost wakeup becomes a failure, not a hang.
A clear message	You know what broke without a debugger.

Reading an LLM test: five questions

1. Do the threads really run at the same time?
2. Does the timing change from try to try?
3. How many tries happen per second?
4. Can the test always stop itself?
5. Does it pass on the correct code?

If you only check one thing, check number 5.

A lost wakeup in Taskflow v3.8 - v3.11



Signals are counted, not owned. Waiter 0 used up the signal meant for Waiter 1.

- NonblockingNotifierV2, the default notifier from v3.8 until v4.0.
- Found by claude_N2_2b_2_s4.cpp: fails 10 of 10 on v3.9 and v3.11, passes 10 of 10 on today's notifier.

Results Summary

- Taskflow’s 194 existing tests missed A1, A2, A5, and N1, even after 20 runs on both architectures.
- Our generated test detected A1 immediately in all 5 trials.
- Detection speed varied significantly across models.
- For A5, one GPT-5.5 test needed 41,531 iterations, while Claude detected it immediately.

Lessons Learned

- Ten out of ten bugs. Not ten out of ten tests.
- Always validate tests against the clean implementation.
- Different prompts caught different bugs.
- Memory-ordering bugs were more detectable than I expected.
- Injecting realistic bugs was harder than generating tests.

Next Steps

- Build a reproducible workflow leveraging agentic models.
- Sample the same bugs and pass it through the workflow.
- Observe if the accuracy of detecting bugs increases.

Start using Taskflow today!

Acknowledgements

Dr. Tsung-Wei (TW) Huang

University of Wisconsin-Madison’s Open Source Program Office (OSPO)

CppCon Team

Thank you! :)